Building Open Factory Studio

What you need

Tool	Version
CMake	3.26 or newer
Ninja	Any recent version
C++ compiler	One with C++20 support: Visual Studio 2022, clang 17 or GCC 12
Qt	6.8 or newer, with the Core5Compat, Svg and Image Formats modules
Python	3.8 or newer (only for the scripts in the tools folder)

CMake downloads the other libraries the first time you configure the project, so you need an internet connection for that first step. A few small libraries are already in the third_party folder. Qt's Linguist tools are only needed if you work on translations.

Installing Qt

The easiest way is aqtinstall. It works on every system and doesn't need admin rights.

```
python -m pip install aqtinstall

# Windows with llvm-mingw (comes with a matching compiler)
aqt install-qt windows desktop 6.8.3 win64_llvm_mingw -m qt5compat qtimageformats -O C:/Qt
aqt install-tool windows desktop tools_llvm_mingw1706 qt.tools.win64_llvm_mingw1706 -O C:/Qt

# Windows with Visual Studio 2022
aqt install-qt windows desktop 6.8.3 win64_msvc2022_64 -m qt5compat qtimageformats -O C:/Qt

# Linux
aqt install-qt linux desktop 6.8.3 linux_gcc_64 -m qt5compat qtimageformats -O ~/Qt

# macOS
aqt install-qt mac desktop 6.8.3 clang_64 -m qt5compat qtimageformats -O ~/Qt
```

The normal Qt installer from qt.io works too.

Building

Tell CMake where Qt is with the OFS_QT_DIR environment variable. If you use llvm-mingw, also set OFS_LLVM_MINGW to the compiler folder.

Preset	What to set
win-llvm-mingw	OFS_QT_DIR=C:/Qt/6.8.3/llvm-mingw_64 and OFS_LLVM_MINGW=C:/Qt/Tools/llvm-mingw1706_64
win-msvc	OFS_QT_DIR=C:/Qt/6.8.3/msvc2022_64, and run it from the "x64 Native Tools" prompt
linux	OFS_QT_DIR=~/Qt/6.8.3/gcc_64

Preset	What to set
macos	OFS_QT_DIR=~/Qt/6.8.3/macos

```
cmake --preset win-llvm-mingw
cmake --build --preset win-llvm-mingw
ctest --preset win-llvm-mingw
```

Each preset also has a release version, for example win-msvc-release. If you don't want to set the variables every time, make a CMakeUserPresets.json file in the project folder. Git ignores it. For example:

```
{
 "version": 6,
 "configurePresets": [{
  "name": "dev", "inherits": "win-llvm-mingw",
  "environment": { "OFS_QT_DIR": "C:/Qt/6.8.3/llvm-mingw_64",
                   "OFS_LLVM_MINGW": "C:/Qt/Tools/llvm-mingw1706_64" }
 }],
 "buildPresets": [{ "name": "dev", "configurePreset": "dev" }],
 "testPresets": [{ "name": "dev", "inherits": "win-llvm-mingw",
                   "configurePreset": "dev" }]
}
```

Build options. Turn them on or off with -D<option>=ON or OFF:

Option	Default	What it does
OFS_BUILD_TESTS	ON	Builds the tests (downloads Catch2)
OFS_BUILD_TOOLS	ON	Builds the command line tools, like ofs-sffdump
OFS_WITH_RAR	ON	Adds RAR support (downloads and builds libarchive)
OFS_WERROR	OFF	Treats warnings as errors (the CI build turns it on)
OFS_SANITIZERS	OFF	Turns on the address and undefined behavior sanitizers (clang and GCC only)

Running it

The build folder is laid out like an installed copy, with the program in bin.

- Windows: run cmake --build --preset <preset> --target deploy-dev once. It copies the Qt files and the C++ runtime next to bin/ofs.exe. After that you can start it directly.
- Linux and macOS: bin/ofs runs as it is.

A few environment variables are useful while working on it:

Variable	What it does
OFS_DATA_DIR	Uses another data folder (handy for editing presets while the program runs)
OFS_CONFIG_DIR	Uses another settings folder (the tests use this)
OFS_CORPUS_KFM	Folder with a stock Kung Fu Man, turns on the corpus tests
OFS_CORPUS_MUGEN	Your M.U.G.E.N folder, turns on the corpus and oracle tests

Tests with real game files

The normal test run skips the tests that need real game files. To run them, set the variables above and run them on their own:

```
export OFS_CORPUS_KFM="C:/path/to/chars/kfm"
export OFS_CORPUS_MUGEN="C:/path/to/mugen-1.0"
ctest --test-dir build/<preset> -L "corpus|oracle" --output-on-failure
```

The oracle tests use sff2png.exe and sprmake2.exe from your M.U.G.E.N folder, so they only run on Windows. Checking a whole M.U.G.E.N folder takes a few minutes in a debug build. A release build is much faster.

To look inside SFF and SND files, use bin/ofs-sffdump (info, list, extract, convert and snd-list).

Common problems

- Don't mix the two MinGW versions. Qt's win64_mingw is built with GCC and win64_llvm_mingw with clang. Use the compiler that matches your Qt download, or the build won't link or will crash when it starts.
- Windows can pick up the wrong libc++.dll, libstdc++-6.dll or Qt files if another program put them earlier in PATH. The deploy-dev step copies the right ones next to the program, and Windows looks there first.
- C++20 module scanning is off. The project doesn't use modules, and the scan fails when clang-scan-deps and the compiler are different versions.

Making packages

To install it somewhere, run cmake --install build/<preset> --prefix <folder>. The release packages are made by the scripts in the packaging folder:

- Windows: pwsh packaging/windows/package.ps1 -Preset win-msvc-release (the installer needs Inno Setup 6 or 7; without it you only get the portable zip)
- Linux: packaging/linux/package.sh linux-release (needs linuxdeploy and its Qt plugin)
- macOS: packaging/macos/package.sh macos-release